

WST AI Engineering — Daily Report

Project 6: Workshop Management & Student Practical Training · Badr University DevHub · Role: AI Engineer

1. Summary

Today's work covered the AI/data-science scope of requirement WST-FR-14 (parts-demand and training-risk flags): environment setup, two rule-based baselines, a local PostgreSQL database seeded with synthetic data, two ML experiments each compared against its baseline with proper metrics, full prediction logging per the schema's AI release gate, and cleanup of an earlier prototype that did not meet the brief's requirements.

2. Environment Setup

- Created project folder **wst-ai** with a Python virtual environment (venv).
- Installed core dependencies: psycpg2-binary, pandas, numpy, python-dotenv, scikit-learn.
- Initialized a git repository with a proper .gitignore (excludes venv/, .env, __pycache__).
- Set up db.py for PostgreSQL connections, configured via a local .env file (never committed).

3. Rule-Based Baselines (Required First, per the brief)

The brief requires every AI feature to have a deterministic, explainable baseline before any model is introduced. Two baselines were built directly against the real schema (not mock data):

reorder_baseline.py

Reads current stock levels from stock_balances and recent consumption from stock_movements (type = 'ISSUE'). Flags a part for reorder when available stock (on_hand minus reserved) falls to or below min_level, and suggests a quantity to bring stock back to max_level.

training_risk_baseline.py

Computes three signals per student from attendance_records and assessments: absence rate, count of pending sign-offs, and count of weak results (FAIL / NEEDS_IMPROVEMENT). Combines them into a LOW / MEDIUM / HIGH risk level with a plain-language explanation for each flag raised.

4. Local Database and Synthetic Seed Data

- Set up a local PostgreSQL 14 database (wst_dev) and loaded the team's schema.sql — applied cleanly with no errors.
- Loaded a teammate's existing seed file (wst_seed_data.sql: 100 students, 100 customers, 100 vehicles).
- Wrote seed_data.py as an **additive** generator: it reuses the existing students/customers/vehicles and only adds inventory (parts, stock movements) and training data (courses, sessions, attendance, assessments) on top — nothing existing is overwritten.
- Student risk profiles were deliberately varied (low / medium / high) across the seeded population so the risk model would have a meaningful signal to learn from.

5. ML Experiments (Model vs. Baseline)

Per the brief's AI release gate, no model is presented without a baseline comparison, a metric, and documented limitations. Both experiments log every prediction — baseline and model — to the predictions and prediction_runs tables with an explanation, source, and version.

reorder_forecast_experiment.py — Linear Trend vs. Moving Average

Model: a simple linear regression (numpy.polyfit, degree 1) fit on each part's weekly consumption trend, using a time-based train/test split (never shuffled, since this is a forecasting problem). Compared against the moving-average baseline using MAE and WAPE.

Result: baseline MAE 0.733 vs. model MAE 1.280 — the baseline won on 4 of 5 part/store combinations. This is an expected and valid outcome: the seeded consumption data is fully random, so there is no real trend for a linear model to learn. On real production data, results would likely differ.

training_risk_experiment.py — Logistic Regression vs. Rule Baseline

Model: scikit-learn LogisticRegression trained on three non-protected features only (absence_rate, pending_assessments, weak_results), evaluated with a stratified train/test split.

Documented limitation: no independently-observed "did the student actually fail" outcome exists in this small synthetic dataset. A transparent proxy target (completion_rate < 0.5) was used and flagged clearly in the code's docstring, to be replaced with a real outcome once enough historical terms exist.

Result: baseline precision 0.579 / recall 1.0 vs. model precision 0.909 / recall 0.909 — the model outperformed the baseline on this metric, because the seeded risk signals (unlike the inventory data) were generated with deliberate, learnable patterns.

6. Schema Defect Found and Fixed

While logging the first baseline run, the experiment failed with:

```
psycopg2.errors.StringDataRightTruncation: value too long for type character varying(12)
```

Root cause: in the original schema.sql, the source_kind column on both predictions and prediction_runs is declared as VARCHAR(12), but its own CHECK constraint allows the value 'RULE_BASELINE' — which is 13 characters, one longer than the column allows. This is an internal contradiction in the schema itself, not an application bug.

Fix applied: widened both columns to VARCHAR(20) locally (ALTER TABLE ... ALTER COLUMN source_kind TYPE VARCHAR(20)), and prepared a corrected schema.sql (two-line diff) to send to the team so the shared schema source of truth is corrected.

7. Prototype Cleanup

An earlier prototype (generated by another AI tool) was reviewed and found unsuitable: it trained on CSV files containing a pre-fabricated target column rather than real schema data, and included no baseline comparison at all — both are direct violations of the brief's AI release gate. Those files (app.py, train_models.py, *.joblib models, CSVs, wst_ai.db) were removed from git tracking and excluded via .gitignore, kept locally only for reference.

8. Deliverables (Git History)

Commit	Description
1142149	Initial setup: venv, db connection, reorder baseline draft
e87ec22	Add training-risk baseline
82d368b	Add synthetic seed data generator
9690a63	Add reorder + training-risk ML experiments with baseline comparison and predictions logging
01de83a	Remove prototype artifacts from tracking (flawed target, kept locally as reference only)
348bd27	Add README documenting setup, run order, and known limitations

9. Open Items (Pending the Team)

- Real staging database credentials — all testing so far was done on a local wst_dev database.
- Confirmation that the schema.sql source_kind fix has been applied to the shared/canonical schema.
- Backend/API integration to expose predictions results to the frontend.